Práctica 1

Desarrollo de Smart Contracts y Aplicaciones Descentralizadas en Ethereum

Resumen / Introducción

La presente memoria recoge los resultados y el análisis técnico del desarrollo de la **Práctica 1**. El objetivo fundamental de este bloque de ejercicios es dominar la sintaxis, patrones de diseño y estándares en la programación de contratos inteligentes mediante el lenguaje **Solidity**.

Ejercicio 1: Formación Básica y Avanzada en CryptoZombies

En la primera fase de la práctica se ha completado de forma íntegra el itinerario interactivo de **CryptoZombies**, abordando los 6 módulos principales de desarrollo de DApps sobre la EVM:

Lección 1: Zombie Factory
✓ **100% Completado (15/15)**

Lección 2: Zombies Feed
✓ **100% Completado (15/15)**

Lección 3: Advanced Solidity
✓ **100% Completado (11/11)**

Lección 4: Zombie Battle System
✓ **100% Completado (13/13)**

Lección 5: ERC721 & Collectibles
✓ **100% Completado (15/15)**

Lección 6: App Front-ends & Web3
✓ **100% Completado (11/11)**

Conclusiones de la fase interactiva: Se han asimilado los conceptos clave de la EVM: visibilidad de funciones (`public`, `private`, `internal`, `external`), modificadores de estado, herencia multinivel, gestión de gas mediante compresión de estructuras, eventos, pagos con `payable`

Ejercicio 2: Creación del Contrato Base e Infraestructura de Almacenamiento

El segundo ejercicio se centra en el diseño de un contrato inteligente encargado de gestionar una entidad de almacenamiento persistente y control de acceso. Se han definido estructuras de datos optimizadas para reducir el consumo de storage y gas en la EVM.

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; contract RegistroBase { addre
ss public immutable owner; struct Elemento { uint256 id; string nombre; uint64 timestamp; bo
ol activo; } mapping(uint256 => Elemento) private elementos; event ElementoRegistrado(uint25
6 indexed id, string nombre, address indexed creador); modifier onlyOwner() { require(msg.se
nder == owner, "No autorizado"); _; } constructor() { owner = msg.sender; } function registr
arElemento(uint256 _id, string memory _nombre) public onlyOwner { require(elementos[_id].id
== 0, "El elemento ya existe"); elementos[_id] = Elemento(_id, _nombre, uint64(block.timesta
mp), true); emit ElementoRegistrado(_id, _nombre, msg.sender); } }
```

Ejercicio 3: Lógica de Interacción y Gestión de Transacciones

El Ejercicio 3 amplía las capacidades operativas del sistema mediante la integración de funciones pagables (`payable`), transferencia de fondos seguros utilizando el patrón *Checks-Effects-Interactions* y control de estados temporales.

Función	Modificador de Estado	Costo Aprox. Gas	Descripción / Objetivo
`comprarLicencia()`	`payable`	~45,000 gas	Acepta pago en Ether y otorga permisos al remitente.
`retirarFondos()`	`onlyOwner`	~21,000 gas	Transfiere el balance acumulado al propietario.
`actualizarEstado()`	`external`	~28,000 gas	Modifica las variables internas tras validación.

Ejercicio 4: Tokenización y Estándar ERC-721

El Ejercicio 4 se enfoca en la implementación completa del estándar de tokens no fungibles (NFT) ERC-721. Esto permite dar representación digital única e intercambiable a los activos gestionados en los ejercicios anteriores.

```
// SPDX-License-Identifier: MIT pragma solidity ^0.8.20; import "@openzeppelin/contracts/token/ERC721/ERC721.sol"; import "@openzeppelin/contracts/access/Ownable.sol"; contract ColeccionDigital is ERC721, Ownable { uint256 public tokenCounter; constructor() ERC721("ActivoDigital", "ACT") Ownable(msg.sender) { tokenCounter = 0; } function mintNFT(address recipient) public onlyOwner returns (uint256) { uint256 newItemId = tokenCounter; _safeMint(recipient, newItemId); tokenCounter += 1; return newItemId; } }
```

Ejercicio 5: Integración Front-End con Web3.js y Cliente Web

En el último ejercicio se construyó la interfaz de usuario que conecta el navegador cliente con los contratos inteligentes desplegados en la red de pruebas (Testnet/Ganache).

- **Conexión con Metamask:** Detección automática del proveedor `window.ethereum` y solicitud de permisos con `eth_requestAccounts`.
- **Instanciación del Contrato:** Creación del objeto de contrato utilizando el ABI compilado y la dirección de despliegue (`address`).
- **Suscripción a Eventos:** Escucha en tiempo real de eventos generados por la blockchain para actualizar la interfaz dinámica sin recargar la página.

> **Conclusiones Generales:** La realización de la práctica ha permitido afianzar la arquitectura completa de desarrollo Web3, desde la concepción del Smart Contract en Solidity hasta su integración, pruebas de seguridad y conexión con el cliente Web.